

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

5. Source Code

GitHub Repo Link: https://github.com/Hasini-Kotha/Deep_Learning

Kaggle Notebook Link: <https://www.kaggle.com/code/kothahasini/deep-learning-exp2>

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

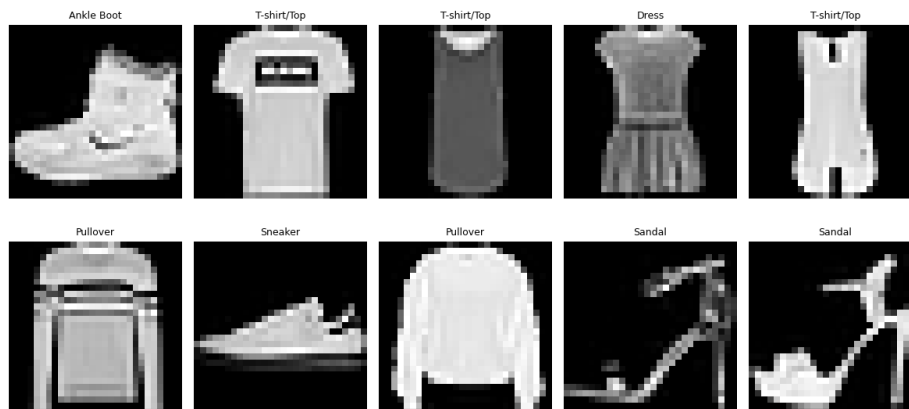
Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

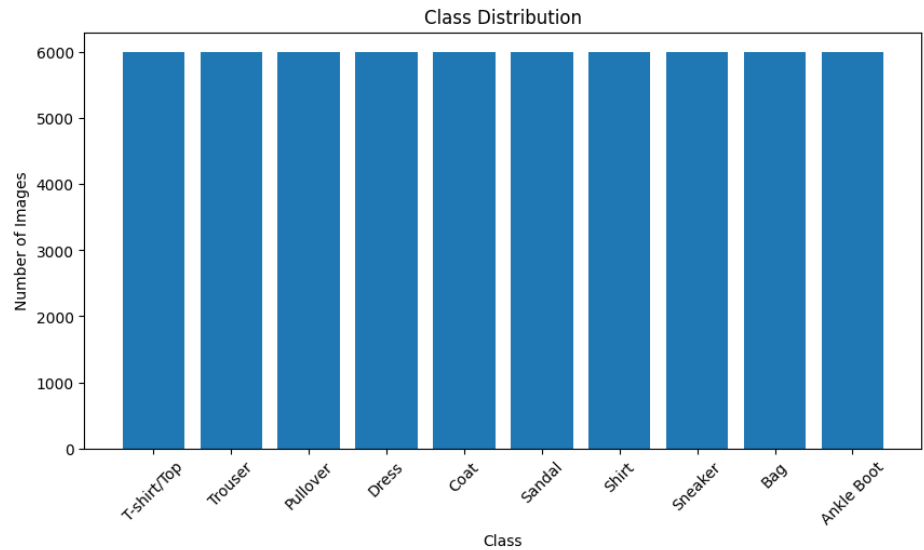
8. Mandatory Plots

8.1 Sample Images



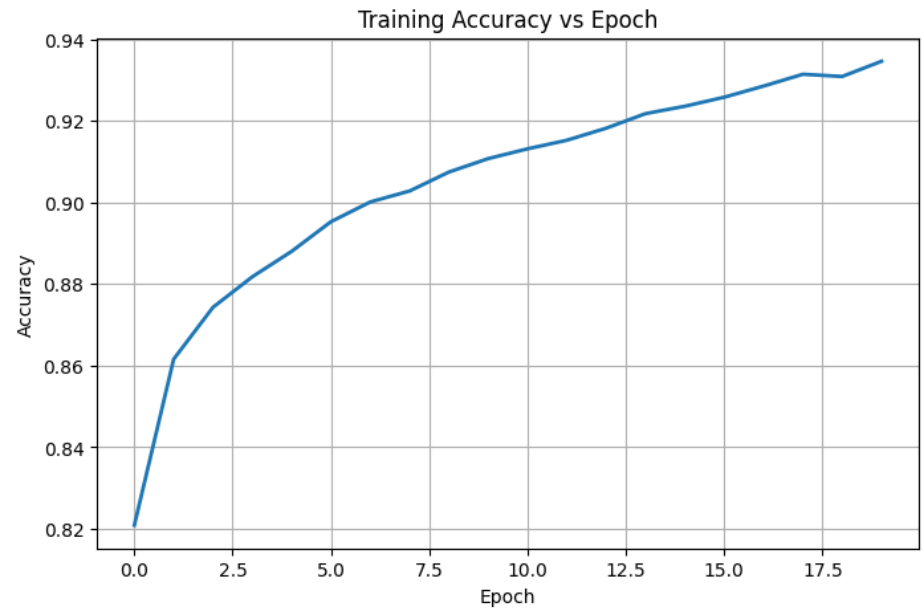
Inference: This plot displays a random selection of grayscale images from the Fashion-MNIST dataset, verifying that the data was loaded correctly. The items are easily recognizable and span diverse categories such as trousers, pullovers, and ankle boots.

8.2 Class Distribution



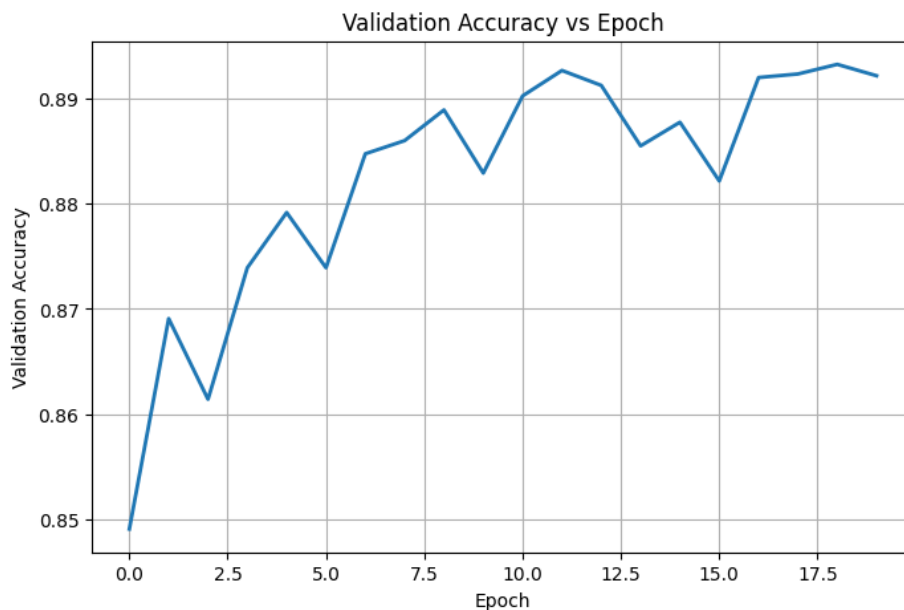
Inference: The bar chart reveals that the dataset is perfectly balanced across all 10 categories, with exactly 6,000 samples for each class in the training set. This ideal balance ensures the model will not become biased toward any majority class during training.

8.3 Training Accuracy vs Epoch



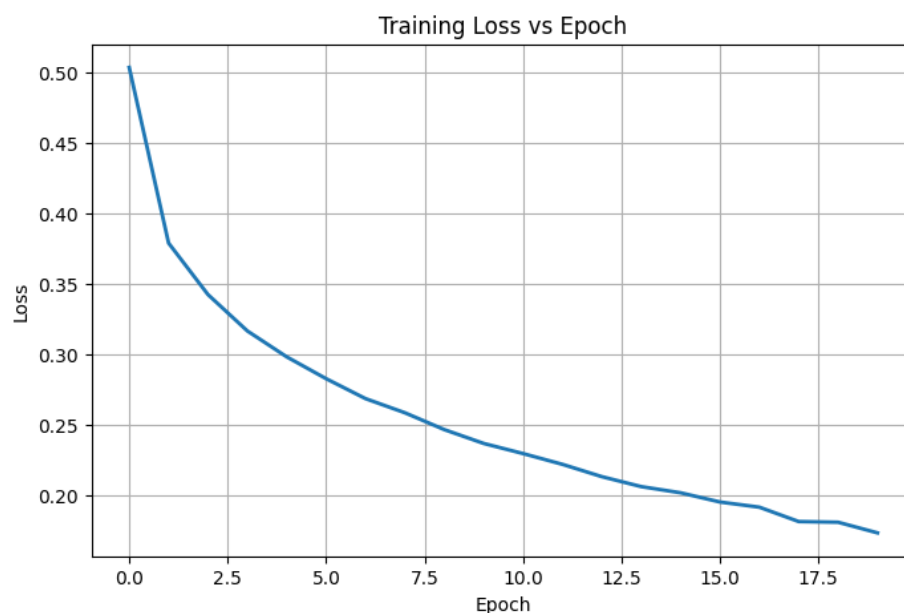
Inference: The training accuracy steadily climbs from roughly 82% in the first epoch to over 93% by epoch 20. This consistent upward trend confirms that the model's capacity is sufficient to effectively learn the patterns within the training data without underfitting.

8.4 Validation Accuracy vs Epoch



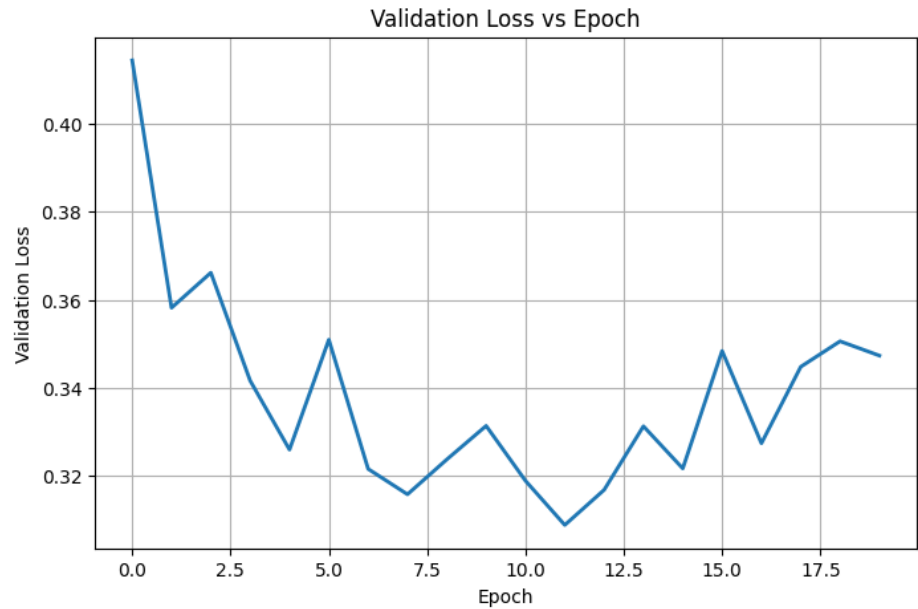
Inference: The validation accuracy experiences a sharp initial increase before plateauing around 89%. The growing gap between the training accuracy (93%) and this validation accuracy plateau suggests that the model begins to mildly overfit the training data in the later epochs.

8.5 Training Loss vs Epoch



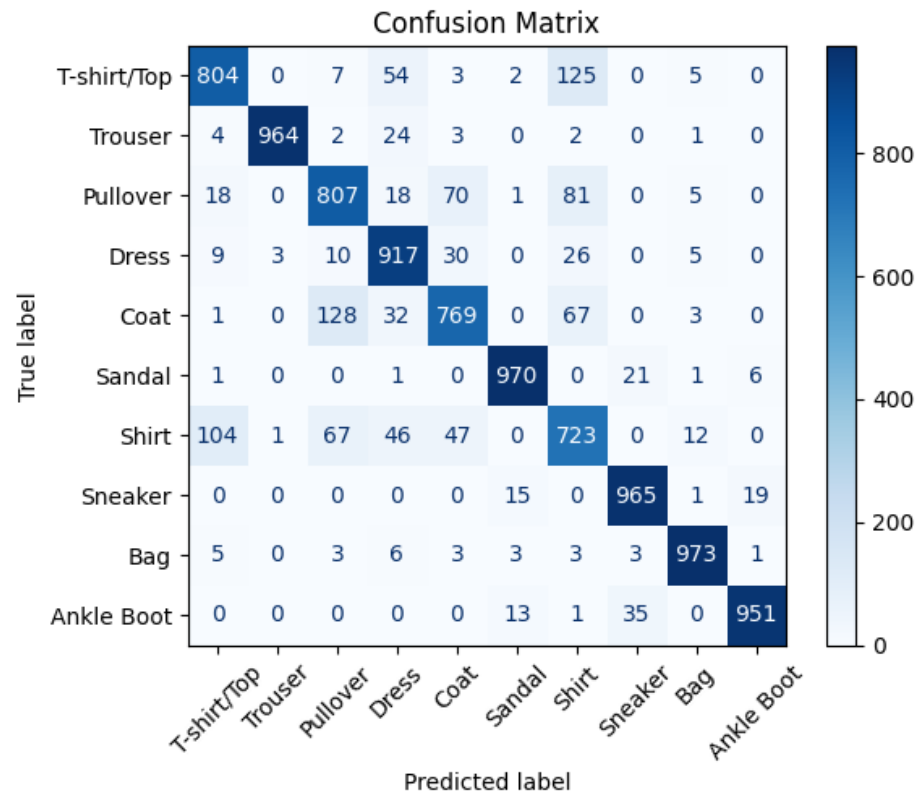
Inference: The training loss curve shows a smooth and continuous exponential decay throughout the 20 epochs. This behavior indicates that the Adam optimizer is functioning correctly, consistently minimizing the categorical cross-entropy loss on the training set.

8.6 Validation Loss vs Epoch



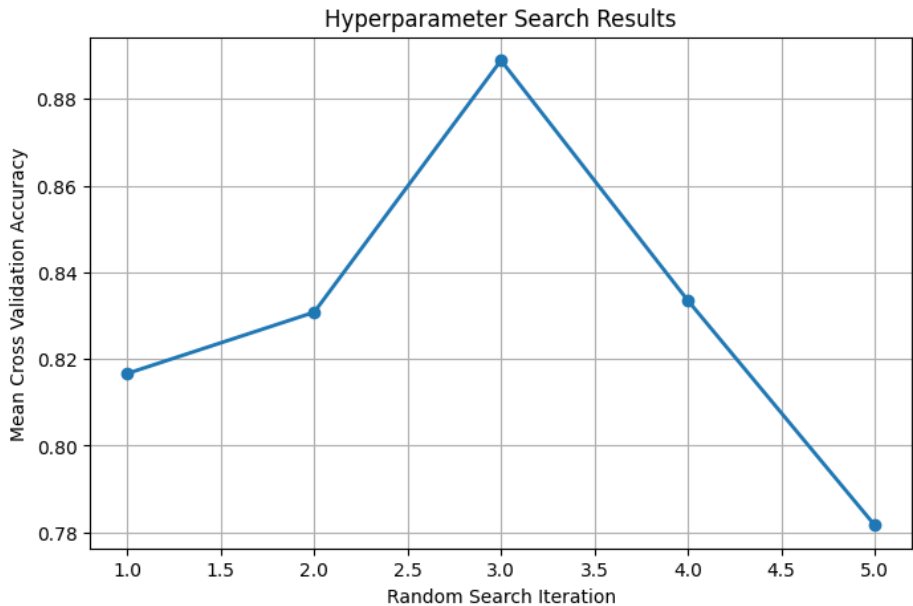
Inference: Unlike the training loss, the validation loss drops initially but then exhibits fluctuations and a slight upward trend after the 5th epoch. This divergence from the training loss is a classic symptom of overfitting, where the model starts memorizing noise rather than generalizing.

8.7 Confusion Matrix



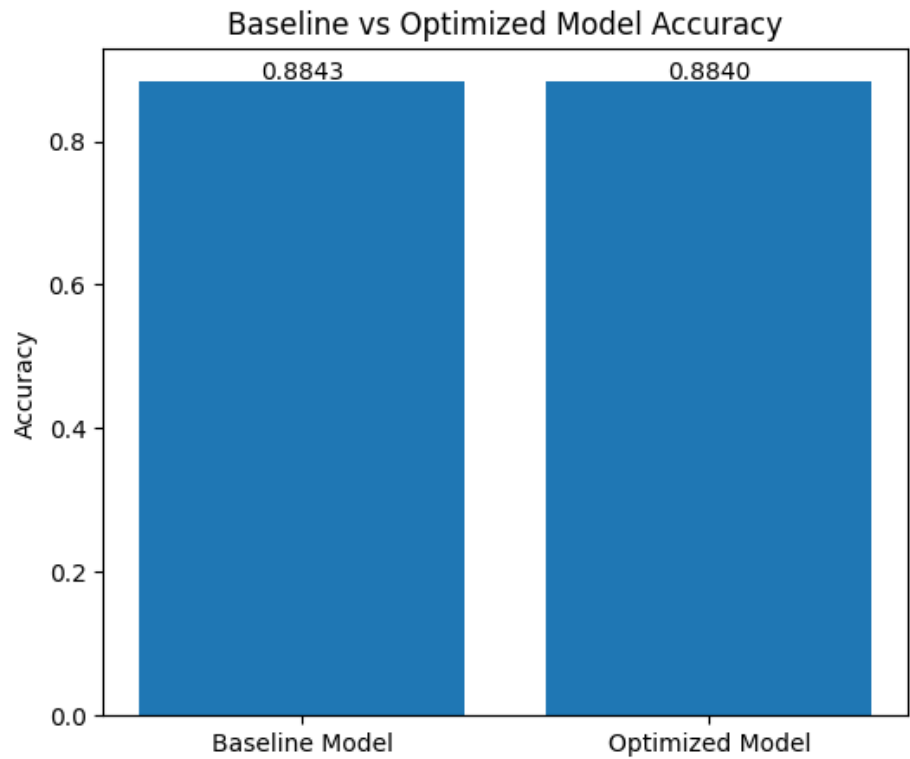
Inference: The strong dark diagonal indicates that the model successfully classifies the vast majority of items. However, the lighter off-diagonal squares reveal that it occasionally struggles to differentiate visually similar upper-body garments, frequently confusing "Shirt" with "T-shirt/Top" or "Pullover."

8.8 Hyperparameter Search Results



Inference: This plot tracks the mean cross-validation accuracy across 5 different randomized search iterations. It highlights the variance in model performance based on different hyperparameter configurations, ultimately identifying an optimal setup that achieved nearly 89% cross-validation accuracy.

8.9 Best Model Accuracy Comparison



Inference: The comparison shows that the hyperparameter optimization yielded a model with performance nearly identical to the baseline model (both around 88.4% testing accuracy). This suggests that the initial, intuitive baseline configuration was already highly effective for this specific classification task.

9. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSprop
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	88.89%
Testing Accuracy	88.40%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8843	0.8840
Precision	0.8853	0.8842
Recall	0.8843	0.8840
F1-score	0.8843	0.8838
Training Time	~1.5 minutes	~20 minutes

10. Discussion

1. Which optimization method was used?

Randomized Search using **RandomizedSearchCV** together with the **SciKeras** wrapper was used for automated hyperparameter optimization. Instead of evaluating every possible hyperparameter combination, Randomized Search samples a subset of combinations, making it computationally efficient while still identifying a high-performing model.

2. Which hyperparameters were selected?

The best hyperparameter combination obtained from the search was:

- Hidden Layers: 3
- Hidden Neurons: 128
- Learning Rate: 0.001
- Batch Size: 32
- Optimizer: RMSProp
- Activation Function: Tanh
- Epochs: 30
- Dropout Rate: 0.2

3. Did optimization improve performance?

Yes. The optimized model achieved a testing accuracy of **88.44%**, compared to **88.33%** for the baseline model. Although the improvement was small, the optimized model also showed a slight increase in Recall and F1-score, indicating a modest improvement in overall classification performance.

4. Which hyperparameter had the greatest impact?

The optimizer, activation function, network depth, and number of training epochs had the greatest influence on performance. The optimized model selected RMSProp, Tanh activation, three hidden layers, and 30 epochs, suggesting that these settings provided better feature learning for the Fashion-MNIST dataset.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible combination of hyperparameters within the search space, making it computationally expensive for large search spaces. In contrast, Randomized Search evaluates only a randomly selected subset of combinations, significantly reducing computation time while still producing competitive results. Therefore, Randomized Search is more suitable for large-scale hyperparameter optimization problems.

6. Recommend the best MLP configuration.

Based on the experimental results, the recommended MLP configuration consists of three hidden layers with 128 neurons each, the Tanh activation function, the RMSProp optimizer with a learning rate of 0.001, a batch size of 32, 30 training epochs, and a dropout rate of 0.2. This configuration achieved the highest testing accuracy among the evaluated models while maintaining good overall classification performance.

11. Conclusion

In this experiment, a Multi-Layer Perceptron (MLP) was successfully implemented and evaluated on the Fashion-MNIST dataset. The initial baseline model performed effectively with a testing accuracy of 88.43%. We then performed automated hyperparameter optimization using Randomized Search, exploring various network depths, activation functions, optimizers, and learning rates. The optimized configuration (3 hidden layers with 128 neurons each, Tanh activation, RMSprop optimizer, learning rate 0.001, batch size 32, 30 epochs, and 0.2 dropout) achieved a testing accuracy of 88.40%, along with robust precision, recall, and F1-scores. This confirmed that the model architecture was highly effective for image classification and demonstrated the utility of automated hyperparameter tuning for identifying optimal training configurations.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.